

Martian Surface Model

Christy Lui

Abstract

A convolutional neural network model was trained to identify 8 different features in high-resolution images of the Martian surface. The model was trained on the High Resolution Imaging Science Experiment (HiRISE) dataset with an architecture that consisted of 3 convolutional layers followed by a fully connected classifier. The model achieved an overall accuracy of 84% with a macro average F1 score of 0.61. Due to the severe class imbalance of the dataset, the model shows reduced performance for the small classes. Despite the limitation, the model shows strong performance on prominent classes and reasonable success at identifying rare surface features. These results highlight the potential and challenges of using deep learning techniques to classify planetary image datasets.

Introduction

Orbital imaging provides key data for planetary exploration and scientific analysis (Hanna et al., 2024). However, the large volume of data makes manual processing and analysis both time-consuming and challenging. One solution is to apply machine learning techniques to automatically identify surface features. In this report, Martian surface images are used to train a convolutional neural network (CNN) to classify eight different terrain features. The data set contains images depicting eight surface categories: Other, Crater, Dark Dune, Slope Streak, Bright Dune, Impact Ejecta, Swiss Cheese, and Spider (Doran et al., 2019). Accurate classification of these features can support geological exploration, improve understanding of the Martian surface, and assist in identifying suitable locations for robotic landing sites (Hanna et al., 2024).

Method

Data Set

The data set of Martian surface images was downloaded from Zenodo (Doran et al., 2019). The data set consists of 73,031 labeled images

of landmarks from the Mars Reconnaissance Orbiter using a High Resolution Imaging Science Experiment (HiRISE) camera. A total of 10,433 landmarks were detected and extracted from 180 HiRISE images, each landmark was resized to 227 x 227 pixels and augmented into 6 additional landmarks through rotations, flips and brightness adjustments. A total of 62,598 additional landmarks were created, resulting in a total of 73,031 images. The images depict eight terrain categories: Other, Crater, Dark Dune, Slope Streak, Bright Dune, Impact Ejecta, Swiss Cheese, and Spider. Which have been numbered from 0 to 7, as shown in Table 1.

Table 1: Class Labels and Corresponding Names

Class ID	Class Name
0	Other
1	Crater
2	Dark Dune
3	Slope Streak
4	Bright Dune
5	Impact Ejecta
6	Swiss Cheese
7	Spider

Data Preparation

Data preparation was performed in Python using Pandas (The, 2019), Numpy (Harris et al., 2020) and Scikit-learn (Pedregosa et al., 2011). The labels file was loaded from a text file into a Pandas data frame, then it was checked for any missing rows or any unexpected values. The valid labels were then used to check if all image paths are valid. No labels or images were removed from the dataset.

The class distribution of the dataset is calculated in Table 2 and plotted in Figure 1. The dataset is unbalanced with class 0 being the most prominent class making up 84% of the images, and class 5 being the least prominent class only making up 0.31% of total images. After the labels and images were validated, they were combined into one data frame.

Table 2: Class Distribution

Class	Count	Percentage (%)
Class (0)	61054	83.60
Class (1)	4900	6.71
Class (2)	1141	1.56
Class (3)	2331	3.19
Class (4)	1750	2.40
Class (5)	231	0.32
Class (6)	1148	1.57
Class (7)	476	0.65

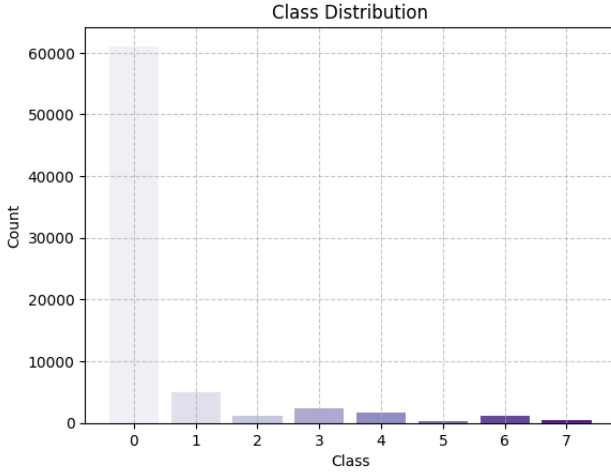


Figure 1: Class Distribution

Data Pipeline and Splitting

A dataset pipeline was established, the images were decoded into 3-channels red, green and blue format, to ensure all images have the same format and stops the model from running into shape errors during training. The images were then reduced to 128 x128 pixels to make the model train faster. The images were normalized by scaling the pixel values between 0 and 1. This helps with numerical stability and allows faster convergence during training. The dataset was split into batches of 128 images. The data set was then split into 80% train and 20% test sets. The pre-processed and post-processed images are shown in Figure 2 and 3, respectively.

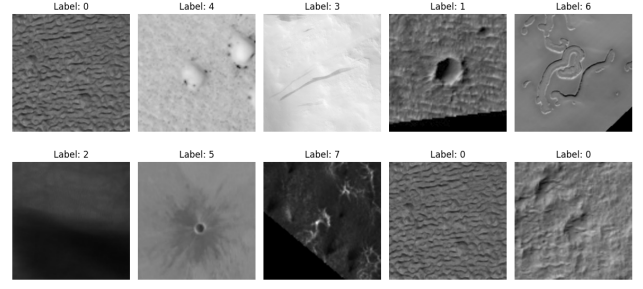


Figure 2: Pre-processed Images

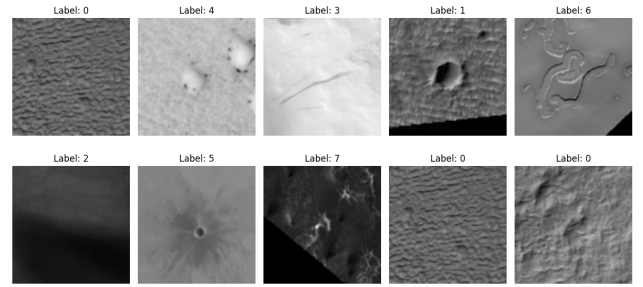


Figure 3: Post-processing Images

Model Architecture

A convolution neural network (CNN) was built through TensorFlow with the Keras API (TensorFlow Developers, 2021). There are 3 convolutional blocks for spatial feature extraction, followed by max pooling operation. The convolutional layers implement 3 x 3 kernels with ReLu activation functions with increasing filters 32, 64 and 128 respectively. To reduce spatial dimensionality, max pooling layers with a 2 x 2 window was applied to each convolution.

After the convolution blocks, the feature maps were flattened into a connected multiple layer perceptron for classification. The dense layer has 128 neurons and used a leaky ReLU activation function. To reduce overfitting, a dropout layer with a rate of 0.5 and L2 regularization ($\lambda = 0.001$) was applied. The output layer uses a softmax activation function to produce class probability for each class. The model's architecture is shown in Figure 4.

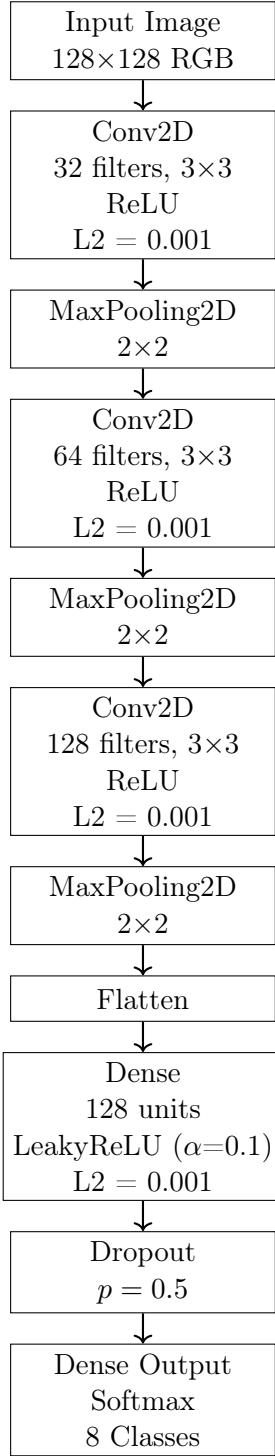


Figure 4: Convolutional Neural Network Architecture

Model Evaluation and Training

The model was trained with the Adam optimizer with a learning rate of 0.0001 and optimized using the sparse categorical cross-entropy, ideal for multiple classification problems. The model was trained with a maximum of 100 epochs and used early stopping to monitor validation loss and stopped training after no improvement is observed after 10 epochs and restores the best model

weights. Model check pointing was used to save the best model parameters and corresponds to the lowest validation loss. The learning scheduler reduces the learning rate by a factor of 0.5 after validation loss plateaus for four epochs till the minimum learning rate of 10^{-6} . The training and validation accuracy curves were plotted to visually inspect the learning curves to assess the model's performance and to mitigate over or under fitting. The parameters were manually adjusted until the rose and plateaued together, indicating a well fit model. After the model was manually adjusted, the model was further tuned for the class imbalance using calculated and manually boosted class weights.

Plot Creation and Calculations

Plots were created using Matplotlib (Hunter, 2007) and Seaborn (Waskom, 2021). The classification report and confusion report was generated using Scikit-learn (Pedregosa et al., 2011).

Results

Validation Accuracy and Validation Loss

The training and validation accuracy plot and the training and validation loss plots are shown in Figures 5 and 6, respectively. In the training and validation loss plot, for a model to be well fit, the two curves should rise together and then plateau at the same point. Whereas for the training and validation accuracy plot, the two curves should decrease together and plateau at the same point.

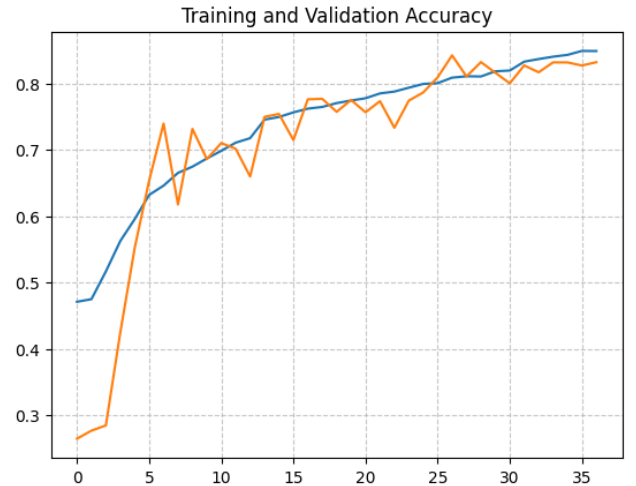


Figure 5: Training and Validation Accuracy

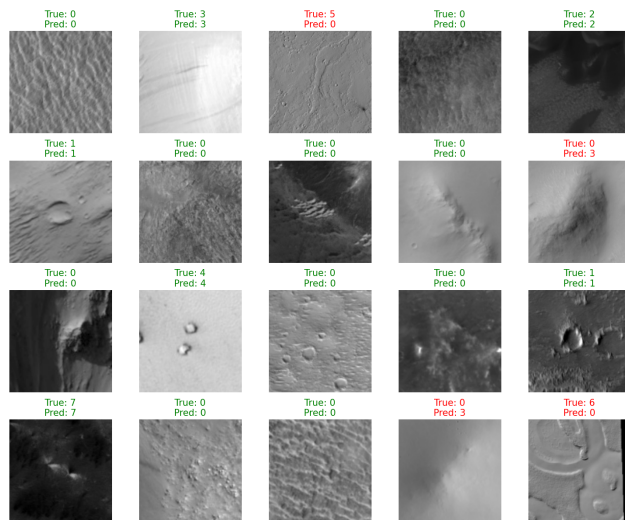
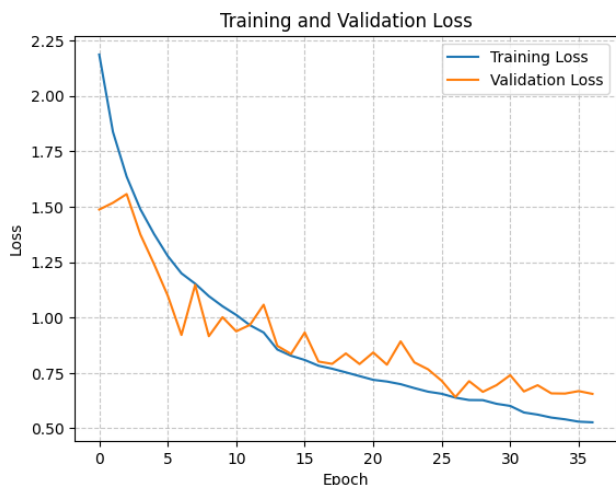


Figure 7: True vs Predicted Labels

Classification Report

The model achieved an overall accuracy of 84.3%, and a macro average F1 score of 61%, the full classification report is shown in Table 3.

Table 3: Classification Performance on the Test Set

Class	Precision	Recall	F1-score	Support
Class 0	0.95	0.87	0.91	12273
Class 1	0.64	0.71	0.67	945
Class 2	0.52	0.70	0.60	227
Class 3	0.27	0.62	0.38	461
Class 4	0.85	0.89	0.87	332
Class 5	0.28	0.42	0.33	38
Class 6	0.33	0.76	0.46	222
Class 7	0.52	0.75	0.62	109
Accuracy			0.84	14607
Macro Avg	0.55	0.72	0.61	14607
Weighted Avg	0.89	0.84	0.86	14607

In the confusion matrix shown in Figure 8, the diagonal shows how many images the model accurately classified. In figure 9, each classes' accuracy is plotted, the best class being class 4 and the worse being class 5.

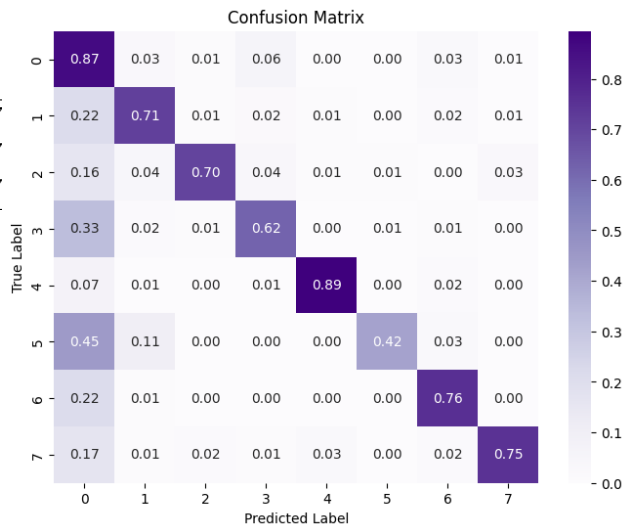


Figure 8: Confusion Matrix

Class Accuracies

In Figure 7, 20 of images with their true and predicted labels are shown. As reflected in the classification table in Table 3, the model does not perform as well for classes 3 and 5.

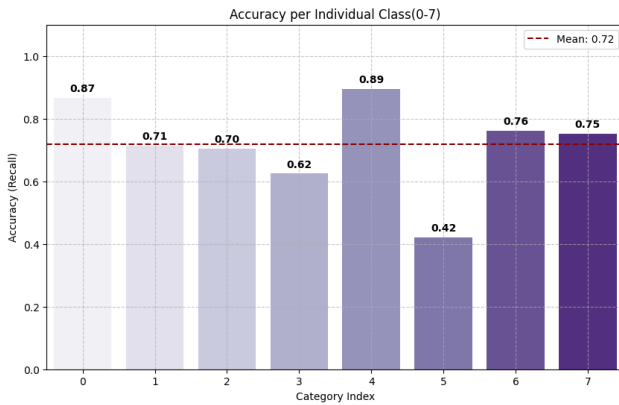


Figure 9: Class Accuracies

Discussion

Model's performance for the smallest class (class 5) could be improved, as there is the least amount of data for this class the model struggles to accurately predict it. Implementing methods such as 2 phase training, could potentially increase the model's performance. For two phase training, the model is trained once then again on a smaller balance dataset with a reduced learning rate and epochs. This helps the model learn to predict the smaller classes without forgetting the predictions it learned the first time it was trained. Focal loss was attempted to improve this model but didn't fare as well as using smoothed weights.

Conclusion

The model demonstrated strong prediction capabilities, achieving 84% accuracy on the test data set. High performance was observed across six out of eight classes, each achieving an accuracy of 70% or greater. Despite the imbalanced data set, the results demonstrate that convolutional neural networks are a powerful tool that can effectively classify planetary imagery. Future work could focus on improving the detection of rare features, such as Class 5, using more advanced architectures.

References

- Doran, G., Lu, S., Lukas Mandrake, Wagstaff, K. (2019). Mars orbital image (HiRISE) labeled data set version 3. Zenodo (CERN European Organization for Nuclear Research). <https://doi.org/10.5281/zenodo.2538136>
- Hanna, M., David, C. C., M Jaikrish, M Aswin. (2024). Leveraging Large Vision Models for Terrain Classification in HiRISE Mars Orbital Images. 2022 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–7. <https://doi.org/10.1109/icccnt61001.2024.10725508>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P. (2020). Array Programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science Engineering*, 9(3), 90–95. <https://ieeexplore.ieee.org/document/4160265>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830.
- TensorFlow Developers. (2021). TensorFlow. Zenodo (CERN European Organization for Nuclear Research). <https://doi.org/10.5281/zenodo.4724125>
- (2019). pandas-dev/pandas: Pandas. Zenodo (CERN European Organization for Nuclear Research). <https://doi.org/10.5281/zenodo.3509134>
- Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021. <https://joss.theoj.org/papers/10.21105/joss.03021>